

Resúmenes

Modulo 4: Fundamentos de Python 1

• Arreglos tridimensionales

Python permite incluir listas dentro de otras listas sin limitar su profundidad. Un arreglo tridimensional utiliza tres índices para organizar información.

• Tu primer función

Las funciones permiten convertir código repetitivo en una unidad reusable. Se definen mediante `def`, y solamente se ejecuta cuando son invocadas. Esto permite realizar cambios en un solo lugar en vez de modificar cada repetición del código.

• Funcionamiento de las funciones

Al invocar una función, Python recuerda el lugar desde donde fue llamada, ejecuta su cuerpo y después regresa al punto posterior a la invocación. Una función debe definirse antes de ser invocada.

• Funciones parametrizadas

Los parámetros permiten que una función reciba información. Los parámetros son variables que pertenecen al alcance de la función.

• Paso de parámetros posicionales

Los argumentos se asignan a los parámetros según su posición. Por esta razón, cambiar el orden puede cambiar los valores recibidos.

• Paso de argumento con palabra clave

Los argumentos pueden enviarse utilizando el nombre del parámetro, por lo que su posición deja de ser importante.

• Combinar argumentos posicionales y de palabra clave
Ambas formas pueden combinarse, pero primero deben colocarse los argumentos posicionales y después los argumentos con palabra clave.

• Instrucción return
Return permite terminar la ejecución de una función y devolver el control al lugar donde fue llamada.

• None
None representa la ausencia de un valor. Una función que no tiene return, devuelve implícitamente un None.

• Listas y funciones
Las listas pueden enviarse como argumentos a funciones. Una función puede recorrer sus elementos, modificarlos según correspondan y devolver un resultado.

• Funciones y su alcance
El alcance indica donde un nombre puede ser reconocido. Los parámetros de una función solamente existe dentro de ella.

• Palabra reservada Global
Global permite indicar que una variable utilizada dentro de una función corresponde a una variable definida fuera de ella.

• Triángulos y Teorema de Pitágoras.
Después de comprobar que los valores forman un triángulo, se puede determinar si es rectángulo usando la relación con el teorema.

• Factoriales

El factorial representa el producto de los números desde 1 al número indicado. La función usa un ciclo para realizar las multiplicaciones.

• Serie Fibonacci

Comienza con 1, 1 y cada elemento posterior se obtiene sumando los dos anteriores.

• Tupla

Es una secuencia inmutable, es decir, sus elementos no pueden modificarse directamente.

Una tupla vacía se crea con (), se puede consultar mediante índices, pero no se puede hacer ninguna modificación.

• Diccionario

Es una estructura de datos mutable, que no es una secuencia. Está formado por claves y valor, o se usan llaves y $\cdot$.

Los operadores `in` y `not in` permite comprobar si una clave existe.

`Keys` permite obtener las claves del diccionario mediante un `for`. También se usa `sorted` para recorrerlas ordenadamente. `Items` permite recorrer simultáneamente sus claves y valores. Se agrega usando `update`.

• Errores

`ZeroDivisionError`: Aparece cuando se intenta realizar una división entre cero.

`ValueError`: Un valor recibido no se acepta para la operación.

`TypeError`: Cuando el tipo de dato no puede usarse en el contexto.

`AttributeError`: Un método no existe.

`SyntaxError`: Se viola la gramática de Python.

Exámenes

PROGRA - Documentos de Go... xEdube Interactive : PE1 -- Mod... x +

edube.org/quiz/python-essentials-1-esp-edube/pe1-module-4-quiz-7

Pregúntale a Gemini

PythonINSTITUTE

« PE1 -- Module 4 Quiz »

Progreso (100%)

Tu puntaje: 12/12

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 4 Quiz100%

Volver a presentar la pruebaRevisar prueba

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go... xEdube Interactive : FP1: Examen... x +

edube.org/quiz/python-essentials-1-esp-edube/fp1-examen-modulo-4

Pregúntale a Gemini

PythonINSTITUTE

« FP1: Examen Módulo 4 »

Progreso (100%)

Tu puntaje: 21/22

95%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 4 Test95%

Volver a presentar la pruebaRevisar prueba

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python

4.3.1.8 LABORATORIO: Día del año: escribiendo y utilizando tus propias funciones

LABORATORIO

Tiempo Estimado

20-30 minutos

Nivel de Dificultad

Medio

Requisitos Previos

LABORATORIO 4.1.3.6
LABORATORIO 4.1.3.7

Objetivos

Familiarizar al estudiante con:

- Proyectar y escribir funciones con parámetros.
- Utilizar la sentencia `return`.
- Construir un conjunto de funciones de utilidad.
- Utilizar las funciones propias del estudiante.

Escenario

Tu tarea es escribir y probar una función que toma tres argumentos (un año, un mes y un día del mes) y devuelve el día correspondiente del año. o devuelve `None` si cualquiera de los argumentos no es válido.

Debes utilizar las funciones previamente escritas y probadas. Agrega algunos casos de prueba al código. Esta prueba es solo el comienzo.

```
1- def is_year_leap(year):
2-     if year % 4 != 0:
3-         return False
4-     elif year % 100 != 0:
5-         return True
6-     elif year % 400 != 0:
7-         return False
8-     else:
9-         return True
10-
11- def days_in_month(year, month):
12-     if year < 1 or month < 1 or month > 12:
13-         return None
14-     days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
15-     if month == 2 and is_year_leap(year):
16-         return 29
17-     return days[month - 1]
18-
19- def day_of_year(year, month, day):
20-     # Validar el mes y el año primero utilizando days_in_month
21-     max_days = days_in_month(year, month)
22-     if max_days is None or day < 1 or day > max_days:
23-         return None
24-
25-     total_days = 0
26-     # Sumar los días de los meses anteriores al actual
27-     for m in range(1, month):
28-         total_days += days_in_month(year, m)
29-
30-     # Sumar los días transcurridos en el mes actual
31-     total_days += day
32-     return total_days
33-
34- print(day_of_year(2000, 12, 31))
```

Console >...

366

Prev

Next

Jairo Haziel
Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python

4.3.1.9 LABORATORIO: Números primos: ¿Cómo encontrarlos?

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Medio

Objetivos

- Familiarizar al estudiante con nociones y algoritmos clásicos.
- Mejorar las habilidades del estudiante para definir y emplear funciones.

Escenario

Un número natural es **primo** si es mayor que 1 y no tiene divisores más que 1 y sí mismo.

¿Complicado? De ninguna manera. Por ejemplo, 8 no es un número primo, ya que puedes dividirlo entre 2 y 4 (no podemos usar divisores iguales a 1 y 8, ya que la definición lo prohíbe).

Por otra parte, 7 es un número primo, ya que no podemos encontrar ningún divisor para él.

Tu tarea es escribir una función que verifique si un número es primo o no.

La función:

- Se llama `is_prime`.
- Toma un argumento (el valor a verificar).
- Devuelve `True` si el argumento es un número primo, y `False` de lo contrario.

Sugerencia: Intenta dividir el argumento por todos los valores posteriores (comenzando desde 2) y verifica el resto: si es cero, tu número no puede ser un número primo; analiza cuidadosamente cuándo deberías detener el proceso.

Si necesitas conocer la raíz cuadrada de cualquier valor, puedes utilizar el operador `**`. Recuerda: la raíz cuadrada de x es lo mismo que $x^{0.5}$.

Complementa el código en el editor.

Ejecuta tu código y verifica si tu salida es la misma que la nuestra.

Datos de prueba

Salida esperada:

2 3 5 7 11 13 17 19

```
1- def is_prime(num):
2-     if num <= 1:
3-         return False
4-     for i in range(2, int(num ** 0.5) + 1):
5-         if num % i == 0:
6-             return False
7-     return True
8-
9- for i in range(1, 20):
10-     if is_prime(i + 1):
11-         print(i + 1, end=" ")
12- print()
```

Console >...

2 3 5 7 11 13 17 19

Prev

Next

Jairo Haziel
Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

4.3.1.10 LAB: Convirtiendo el consumo de combustible

MODULE (36%)SECTION (91%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

- Mejorar las habilidades del estudiante para definir, utilizar y probar funciones.

Escenario

El consumo de combustible de un automóvil se puede expresar de muchas maneras diferentes. Por ejemplo, en Europa, se muestra como la cantidad de combustible consumido por cada 100 kilómetros.

En los EE. UU., se muestra como la cantidad de millas recorridas por un automóvil con un galón de combustible.

Tu tarea es escribir un par de funciones que conviertan l/100km a mpg (milas por galón), y viceversa.

Las funciones:

- Se llaman `liters_100km_to_miles_gallon` y `miles_gallon_to_liters_100km` respectivamente.
- Toman un argumento (el valor correspondiente a sus nombres).

Complementa el código en el editor.

```
1 def liters_100km_to_miles_gallon(liters):
2     # 1 galón = 3.785411784 litros
3     # 1 milla = 1609.344 metros = 1.609344 km
4     # mpg = 100 * 1.609344 / (liters * 3.785411784) => o bien equivalente
5     miles_per_100km = 100 * 1609.344 / 1000
6     gal_liters = 3.785411784
7     miles = 100 / 1.609344
8     gallons = liters / gal_liters
9     return miles / gallons
10
11
12 def miles_gallon_to_liters_100km(miles):
13     km = miles * 1.609344
14     liters = 3.785411784
15     # 100 km / litros consumidos
16     # galones por milla = 1 / miles
17     # litros por milla = (1 / miles) * 3.785411784
18     # litros por 100 km = litros por milla * 100
19     gallons_per_mile = 1 / miles
20     liters_per_mile = gallons_per_mile * 3.785411784
21     liters_per_km = liters_per_mile / 1.609344
22     return liters_per_km * 100
23
24 print(liters_100km_to_miles_gallon(3.9))
25 print(liters_100km_to_miles_gallon(7.5))
26 print(liters_100km_to_miles_gallon(10.0))
27 print(miles_gallon_to_liters_100km(60.3))
28 print(miles_gallon_to_liters_100km(31.4))
29 print(miles_gallon_to_liters_100km(23.5))
```

Console>_

60.31143162393162
31.361944444444444
23.521458333333333
3.900739358761747
7.490910297239915
10.009131205673757

Python INSTITUTE

4.7.2.1 PROYECTO: TIC-TAC-TOE

MODULE (95%)SECTION (95%)

- El primer movimiento es de la máquina: siempre coloca una 'x' en el centro del tablero.
- Todos los cuadros están numerados comenzando con el 1 (observa el ejemplo para que tengas una referencia).
- El usuario ingresa su movimiento introduciendo el número de cuadro elegido. El número debe de ser válido, por ejemplo un valor entero mayor que 0 y menor que 10, y no puede ser un cuadro que ya esté ocupado.
- El programa verifica si el juego ha terminado. Existen cuatro posibles veredictos: el juego continúa, el juego termina en empate, tu ganas, o la máquina gana.
- La máquina responde con su movimiento y se verifica el estado del juego.
- No se debe implementar algún tipo de inteligencia artificial, la máquina elegirá un cuadro de manera aleatoria, eso es suficiente para este juego.

El ejemplo del programa es el siguiente:

```
+-----+
| 1 | 2 | 3 |
+-----+
| 4 | X | 6 |
+-----+
| 7 | 8 | 9 |
+-----+
Ingresa tu movimiento: 1
+-----+
| 0 | 2 | 3 |
+-----+
| 4 | X | 6 |
+-----+
```

```
63- if board[0][2] == sign and board[1][1] == sign and board[2][0] == sign:
64-     return True
65-
66- return False
67-
68- def DrawMove(board):
69-     # La función dibuja el movimiento de la máquina y actualiza el tablero
70-     free = MakeListOfFreeFields(board)
71-     if len(free) > 0:
72-         index = randrange(len(free))
73-         row, col = free[index]
74-         board[row][col] = 'X'
75-
76- # Inicialización del juego
77- board = [['1', '2', '3'], ['4', 'X', '6'], ['7', '8', '9']]
78- turn = True # True para la máquina ('X'), False para el usuario ('O')
79-
80- while True:
81-     DisplayBoard(board)
82-
83-     if VictoryFor(board, 'X'):
84-         print("¡La máquina ha ganado!")
85-         break
86-     elif VictoryFor(board, 'O'):
87-         print("¡Has ganado!")
88-         break
89-     elif len(MakeListOfFreeFields(board)) == 0:
90-         print("¡Empate!")
91-         break
92-
93-     if turn:
94-         DrawMove(board)
95-         turn = False
96-     else:
97-         EnterMove(board)
98-         turn = True
```

Console>_

+-----+
| 1 | 2 | 3 |
+-----+
| 4 | X | 6 |
+-----+

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome